
SWU Erasmus Staff Mobility Portal

Technical Document

Plain technical handoff for local engineering use

Document	Technical system and deployment guide
Audience	Local engineers, technical reviewers, and maintainers
Scope	Current repository state and current deployment path
Version	March 2026

Contents

1 Purpose of this document	2
2 System summary	2
3 Technology stack	2
4 Repository layout	3
4.1 Main folders	3
4.2 Important application subfolders	3
5 Application layout and navigation	3
5.1 Public area	3
5.2 Protected workspace	4
5.3 Main page routes	4
5.4 API route groups	5
6 User roles and access model	5
6.1 Role model	5
6.2 Authentication flow	5
6.3 Authorisation model	5
7 Main functional areas	6
7.1 Registration and account approval	6
7.2 Profile management	6
7.3 Mobility case management	6
7.4 Document handling	6
7.5 Officer review workflow	7
7.6 Reporting and CSV export	7
7.7 Audit logging	8
8 Data model overview	8
8.1 Core entities	8
8.2 Important design choices	8
9 Document storage and private downloads	9
10 Validation and security model	9

10.1 Validation	9
10.2 Current security controls	9
10.3 Current security limits	10
11 Testing and quality	10
12 Local installation on another machine	10
12.1 Prerequisites	10
12.2 Configuration files	11
12.3 First-time local startup	11
12.4 Important environment values	11
12.5 Seed data	11
13 Deployment process on another internal system	12
13.1 What the repository already supports	12
13.2 Current recommended deployment model	12
13.3 Deployment sequence	12
13.4 Production-like commands	13
13.5 Optional demo setup	13
13.6 What is not yet included in the repository	13
14 Operational notes for maintainers	14
14.1 Database	14
14.2 Storage	14
14.3 External temporary access	14
15 Current limitations and scope boundaries	14
16 Recommended technical demo flow	15
17 Conclusion	15

Purpose of this document

This document explains the current Erasmus staff mobility portal from an engineering point of view. It is meant for a local engineer who needs to understand what the website does, how the code is organised, how the main workflows work, and how to run or deploy the system on another machine inside the university.

The document is based on the current repository. It describes what is implemented now. It does not claim that later hosting work, broader integrations, or future workflow extensions are already complete.

System summary

The portal is an internal web application for Erasmus staff mobility administration. It supports three user roles:

- staff users, who maintain their own profile, create and submit mobility cases, upload required documents, and follow review results
- officers, who review submitted cases, add comments, review documents, change case status, archive completed cases, and use reports
- admins, who manage users, approvals, master data, settings, and the audit log

The application is local-first. It runs as a Next.js application with a PostgreSQL database and a private local file store for uploaded documents. Authentication is credentials-based. Data access is handled through Prisma. Automated tests cover the main application layers and the critical cross-role workflow.

Technology stack

Area	Current choice
Frontend framework	Next.js App Router
Language	TypeScript
UI layer	React, Tailwind CSS, local UI components under <code>src/components/ui</code>
Forms	React Hook Form with Zod validation
Authentication	Auth.js with credentials provider and Prisma adapter
Database	PostgreSQL
ORM	Prisma
Document storage	Local filesystem through a storage abstraction
Unit and integration tests	Vitest and React Testing Library
Browser tests	Playwright

The main package and script definitions are in `package.json`. Prisma configuration is in `prisma.config.ts`. Local database startup uses `docker-compose.yml`.

Repository layout

The repository is split into application code, schema and seed files, tests, and operational documentation.

Main folders

Path	Purpose
<code>src/app</code>	Next.js pages, layouts, and API routes
<code>src/components</code>	UI building blocks and feature-specific components
<code>src/lib</code>	Application services, validation, auth helpers, reporting, review logic, storage, and shared utilities
<code>prisma</code>	Prisma schema, migrations, and seed script
<code>tests</code>	Unit, integration, component, and end-to-end tests
<code>docs</code>	Startup, testing, storage, deployment, progress, and user-facing documentation

Important application subfolders

Path	Purpose
<code>src/components/auth</code>	Login, registration, and sign-out UI
<code>src/components/cases</code>	Case forms, case tables, status badges, and document panels
<code>src/components/review</code>	Officer review filters, review forms, and review tables
<code>src/components/reports</code>	Reporting filters, summary tables, export actions, and case lists
<code>src/lib/auth</code>	Authentication, approval, guards, and auth navigation logic
<code>src/lib/profile</code>	Staff profile read and update logic
<code>src/lib/mobility-case</code>	Staff case creation, editing, submission, and detail logic
<code>src/lib/documents</code>	Upload policy, version handling, and download authorization
<code>src/lib/review-workflow</code>	Officer case list, case detail, comments, document review, and status changes
<code>src/lib/reporting</code>	Server-side report aggregation and CSV export generation
<code>src/lib/storage</code>	Storage abstraction and local file driver
<code>src/lib/validation</code>	Zod schemas and validation helpers

Application layout and navigation

Public area

The public area is defined by the root layout in `src/app/layout.tsx`. It includes the top bar, public navigation, language toggle, and footer. The public navigation currently exposes:

- /
- /status
- /login
- /register

When a user is already signed in, the public layout also exposes /dashboard.

Protected workspace

The protected workspace is defined in `src/app/dashboard/layout.tsx`. That layout does four things:

- requires approved authentication before rendering the dashboard surface
- shows a role-aware left navigation
- shows signed-in account context, role, and approval state
- provides a shared header with language toggle and sign-out

Navigation is generated in `src/lib/navigation.ts`. The role filter is explicit. Staff, officer, and admin users do not see the same pages.

Main page routes

Route	Access	Purpose
/	Public	Home page and entry point
/login	Public	Credentials login
/register	Public	Staff self-registration
/pending-approval	Public	Pending-account holding page
/status	Public	Local status page
/dashboard	Approved users	Common protected landing page
/dashboard/profile	Approved users	Personal profile editing
/dashboard/staff	Staff	Staff dashboard
/dashboard/staff/cases/new	Staff	New case form
/dashboard/staff/cases/[caseId]	Staff owner only	Staff case detail page
/dashboard/officer	Officer, Admin	Officer dashboard
/dashboard/officer/cases	Officer, Admin	Review register
/dashboard/officer/cases/[caseId]	Officer, Admin	Review detail page
/dashboard/reports	Officer, Admin	Reporting and CSV exports
/dashboard/admin	Admin	Admin dashboard
/dashboard/admin/users	Admin	User lifecycle management

<code>/dashboard/admin/ master-data</code>	Admin	Master data and settings
<code>/dashboard/admin/ audit-log</code>	Admin	Audit log viewer

API route groups

Route group	Purpose
<code>/api/auth/[...nextauth]</code>	Auth.js handlers
<code>/api/register</code>	Staff registration
<code>/api/profile</code>	Profile updates
<code>/api/staff/cases</code>	Staff case creation and list actions
<code>/api/staff/cases/[caseId]</code>	Staff case update actions
<code>/api/staff/cases/[caseId] /documents</code>	Staff document uploads
<code>/api/review/cases/ [caseId]/comments</code>	Officer comments
<code>/api/review/cases/ [caseId]/documents/review</code>	Officer document review actions
<code>/api/review/cases/ [caseId] /missing-documents</code>	Officer missing-document actions
<code>/api/review/cases/ [caseId]/status</code>	Officer case status transitions
<code>/api/admin/users/[userId]</code>	Admin approval, role, and account actions
<code>/api/admin/master-data</code>	Admin master-data and settings updates
<code>/api/reports/export/...</code>	CSV export endpoints
<code>/api/documents/versions/ [versionId]/download</code>	Private document download route
<code>/api/health</code>	Technical health checks
<code>/api/locale</code>	Language cookie updates

User roles and access model

Role model

User roles are defined in the Prisma schema as `STAFF`, `OFFICER`, and `ADMIN`. User approval state is separate and includes `PENDING`, `APPROVED`, `REJECTED`, and `DEACTIVATED`.

This separation is important because role alone is not enough. A user can have the `STAFF` role and still be unable to use the protected area until the account is approved.

Authentication flow

Auth.js is configured in `src/auth.ts`. The credentials provider validates incoming credentials, delegates the actual credential check to `src/lib/auth/service.ts`, and then enriches the session token with role and approval state.

The authentication service performs:

- email normalisation
- password hash comparison with `bcryptjs`
- approval-state checks
- explicit return codes for invalid credentials, pending approval, rejected account, and deactivated account

Authorisation model

Protected actions are not left to the frontend. The current system uses:

- dashboard-level guards
- role-aware navigation
- server-side checks in protected API routes
- ownership checks for staff-owned cases and documents

This means that staff users cannot access another staff user's case or private file just by changing a URL.

Main functional areas

Registration and account approval

Staff registration is handled through `/register` and `/api/register`. The service in `src/lib/auth/service.ts` creates a pending staff account and records an audit entry.

Admin approval and rejection are handled from `/dashboard/admin/users`. The same service layer updates approval state and writes audit records for those decisions.

Profile management

Profile handling is implemented in `src/lib/profile/service.ts`. The profile model links the user to:

- academic title
- faculty
- department

The profile service loads valid selections, preserves legacy values safely if needed, and validates faculty and department consistency during updates.

Mobility case management

Staff case logic is implemented in `src/lib/mobility-case/service.ts`. A case can be:

- created as a draft
- reopened and edited later
- submitted once required fields are complete

The service writes explicit status-history records. Draft and submit flows are intentionally separate. A draft can be incomplete. Submission requires a valid academic year, mobility type, and date range.

Document handling

Document handling is implemented in `src/lib/documents/service.ts` and `src/lib/validation/documents.ts`.

The current business document types are:

- Mobility Agreement
- Final Certificate of Attendance

Each upload is stored as a new version. The system preserves old versions, stores original filename and metadata, and tracks which version is current.

Officer review workflow

Officer review logic is implemented in `src/lib/review-workflow/service.ts`. Officers can:

- view all cases
- search and combine filters
- open case detail pages
- add comments
- review current document versions
- mark missing documents
- change case status
- archive completed cases

Document review state and case workflow state remain separate on purpose. Rejecting a document does not automatically change the whole case status.

Reporting and CSV export

Reporting is handled in `src/lib/reporting/service.ts`, `src/lib/reporting/csv.ts`, and the export routes under `src/app/api/reports/export`.

Current reporting supports:

- mobility count by academic year
- by faculty
- by department
- by mobility type
- by host country
- by host institution
- by status
- open versus completed
- cases without mobility agreement
- cases without final certificate

Audit logging

Important protected actions are stored explicitly in the `AuditLog` model. This includes user lifecycle actions, case creation and submission, document events, review actions, status changes, and settings changes.

Data model overview

Core entities

The main schema lives in `prisma/schema.prisma`. The most important models are:

- `User`
- `Faculty`
- `Department`
- `AcademicYear`
- `CaseStatusDefinition`
- `SelectOption`
- `MobilityCase`
- `MobilityCaseComment`
- `MobilityCaseStatusHistory`
- `MobilityCaseDocument`

- `MobilityCaseDocumentVersion`
- `AuditLog`
- `UploadSetting`
- `ReportSetting`

Important design choices

- User role and approval state are separate.
- Faculty and department are relational, not free-text.
- Case status history is stored explicitly.
- Document review state is separate from case status.
- Document versions are stored explicitly with a current-version pointer.
- Audit records are explicit rows, not inferred only from timestamps.

Document storage and private downloads

The storage abstraction is in `src/lib/storage/driver.ts`. The current concrete implementation is `src/lib/storage/local-driver.ts`.

The local driver:

- writes files under the configured private storage root
- blocks path traversal by resolving and checking the target path
- keeps files outside the public web root

The download route `/api/documents/versions/[versionId]/download` performs permission checks before reading a file. That means a document is never exposed as a direct public static URL.

Validation and security model

Validation

The repository uses Zod schemas under `src/lib/validation`. Validation exists for:

- login and registration
- profile updates
- master data
- mobility cases
- review actions
- document uploads

Current security controls

Current security-related controls include:

- server-side role and approval checks
- ownership checks for staff case and document access
- password hashing
- private document storage
- guarded download routes
- upload validation for extension, size, declared MIME type, and basic file signatures
- blocking of risky PDF active-content markers
- blocking of unsafe DOCX markers such as macro and embedded-object indicators
- CSV formula neutralisation for exports
- audit logging of protected actions

Current security limits

The current repository does not yet include:

- full antivirus or sandbox-based malware scanning
- password reset or email verification
- infrastructure monitoring and alerting
- multi-instance shared storage

Testing and quality

Testing strategy is documented in `docs/testing-strategy.md`. The repository currently includes:

- unit tests for pure validation and helper logic
- integration tests for services and form behaviour
- component tests for visible UI behaviour
- Playwright browser tests for the main role-based workflows

The most important browser reference is `tests/e2e/critical-portal-journey.spec.ts`. That file exercises the main cross-role workflow from registration through archive and reporting.

Current quality commands:

```
npm run lint
npm run typecheck
npm run test
npm run test:e2e
npm run build
```

Local installation on another machine

Prerequisites

The current local setup expects:

- Node.js 24 or newer
- npm
- Docker Desktop or another Docker runtime
- a machine that can expose PostgreSQL on port 5432

Configuration files

Relevant startup files:

- `.env.example`
- `docker-compose.yml`
- `prisma.config.ts`
- `docs/local-startup.md`

First-time local startup

For a fresh local machine:

```
docker compose up -d
npm install
npm run db:migrate
npm run seed
npm run dev
```

Then open:

`http://127.0.0.1:3000`

Useful local checks:

```
http://127.0.0.1:3000/status
http://127.0.0.1:3000/api/health
```

Important environment values

Variable	Purpose
<code>APP_URL</code>	Base app URL
<code>DATABASE_URL</code>	PostgreSQL connection string

AUTH_SECRET	Auth.js secret, must be strong in real use
AUTH_TRUST_HOST	Host trust setting for Auth.js
STORAGE_DRIVER	Current storage backend, local by default
STORAGE_LOCAL_ROOT	Private storage root for uploaded files
MAX_UPLOAD_SIZE_MB	Upload size limit
ALLOWED_UPLOAD_EXTENSIONS	Allowed upload extensions
DEFAULT_LOCALE	Default interface language

Seed data

The seed script is in `prisma/seed.ts`. It upserts:

- demo admin, officer, and staff accounts
- pending, rejected, and deactivated account examples
- faculties, departments, academic years, statuses, and select options
- upload and report settings
- sample draft and submitted cases

For technical review and testing, running `npmrunseed` is useful. For a real internal rollout with real data, it should be used carefully.

Deployment process on another internal system

What the repository already supports

The repository is already structured for a production-like single-host deployment. It has:

- environment-based configuration
- committed Prisma migrations
- a deployment-style migration command
- a build command
- a production start command
- server-side protected actions

Current recommended deployment model

The current supported deployment model is a single internal machine with:

- PostgreSQL
- the Next.js app process
- private document storage on local disk

This is the same model already used locally, just run in production mode instead of development mode.

Deployment sequence

For a clean deployment on another internal machine:

1. Install Node.js and Docker.
2. Copy the repository.
3. Create a real `.env` from `.env.example`.
4. Set a strong `AUTH_SECRET`.
5. Set `DATABASE_URL` for the target PostgreSQL instance.
6. Set `STORAGE_LOCAL_ROOT` to a private directory on disk.
7. Start the database.
8. Install dependencies with `npm install`.
9. Apply committed migrations with `npm run db:migrate:deploy`.
10. Run `npm run build`.
11. Start the application with `npm run start`.

Production-like commands

```
docker compose up -d
npm install
npm run db:migrate:deploy
npm run build
npm run start
```

Optional demo setup

If the engineering team wants a seeded demonstration instance first:

```
npm run seed
```

That is suitable for local testing or a demo server. It should not be confused with a clean production rollout.

What is not yet included in the repository

The repository does not currently include:

- reverse proxy configuration
- TLS certificate management
- service manager files such as systemd units
- backup scheduling
- central log shipping
- monitoring and alerting setup

An internal engineer can still deploy the application on a single machine, but those operational layers need to be added outside the application repository.

Operational notes for maintainers

Database

Database schema is managed through Prisma migrations. For future changes, the repository already supports:

- development migrations with `npmrun db:migrate`
- deployment migrations with `npmrun db:migrate:deploy`
- schema push with `npmrun db:push`

Storage

Uploaded files are stored privately on local disk. The current implementation is suitable for:

- local development
- local evaluation
- single-host internal deployment

For future multi-host deployment, storage should move to a shared persistent backend. The abstraction is already in place for that change.

External temporary access

If temporary public access is needed without paid hosting, the repository already documents tunnel-based sharing in `docs/external-sharing.md`. That is suitable for review sessions, not for permanent hosting.

Current limitations and scope boundaries

The repository is intentionally limited in a few areas. Current boundaries include:

- no EWP integration
- no public Erasmus portal
- no digital-signature workflow
- no password reset or account recovery
- no bulk officer actions
- no saved views or pagination for large queues
- no non-CSV export formats
- no full malware scanning

Recommended technical demo flow

For a technical presentation to local staff, the clearest route through the system is:

1. show the public home page and language toggle
2. show registration and the pending approval path
3. show admin approval of the new account
4. show the staff profile page
5. create a draft case and then submit it
6. upload a mobility agreement
7. switch to the officer role and review the case
8. reject and then accept a document version
9. move the case through status changes
10. show reporting and CSV export
11. finish on the audit log

That sequence matches the way the application is structured and also matches the critical end-to-end test flow already present in the repository.

Conclusion

The current repository is a complete local v1 baseline for an internal Erasmus staff mobility portal. It contains the full application structure, protected role-based workflows, data model, validation, private document handling, reporting, audit logging, and a deployment path for another internal machine.

For a local engineer, the important conclusion is simple: the system is ready to run on another machine with Node.js, PostgreSQL, Docker, environment configuration, Prisma migrations, and a private storage directory. The current codebase is also organised clearly enough to be maintained and extended without needing a full redesign first.